

Final Report

Equivalences and Properties of Mobile Automata

Adam French

Submitted in accordance with the requirements for the degree of
Computer Science with Mathematics BSc

2024/25

COMP3931 Individual Project

Summary

Mobile Automata are an understudied variant of automata. This report proves that Mobile Automata are able to recognize *Recursively Enumerable* languages, meaning it has the same capability as the Turing Machine. The report introduces tools to visualise Mobile Automata and explains its context in an academic setting.

Acknowledgements

Thank you to my supervisor Noleen Koehler for giving me constant support surrounding ideas for the project and help with L^AT_EX. Thank you to the Wolfram community for the active contributors.

Contents

1	Introduction	1
1.1	Outline	1
1.2	Ethical considerations	1
2	Background Research	2
2.1	Introduction to the Theory of Computation	2
2.1.1	Understanding Problems Through Formal Languages	2
2.1.2	Automata and the Chomsky Hierarchy	2
2.1.3	The Turing Machine and Its Limitations	3
2.1.4	Other Models of Computation	3
2.1.5	The Church–Turing Conjecture	3
2.1.6	Textbook Definition of the Turing Machines	4
2.2	Mobile Automata	4
2.2.1	<i>A New Kind of Science</i>	4
2.2.2	Research on Mobile Automata in the Wolfram Community	6
3	Formalising Mobile Automata	11
3.1	Example of a Mobile Automata	11
3.2	Formal definition of a Mobile Automata	11
3.3	Justification of design choices	13
3.3.1	Similarity to Turing Machine	13
3.3.2	Left-Right Reading	13
3.3.3	Terminating machine	13
4	Implementation	14
4.1	Mobile Automata Visualization Website	14
4.1.1	Motivation	14
4.1.2	Design and Implementation	14
4.1.3	Deployed Website	15
4.2	Mobile Automata Proof	16
4.2.1	Proof Implementation	16
4.2.2	Proof Validation with Busy Beaver	17
5	Results and Evaluation	19
5.1	Results	19
5.1.1	The Mobile Automata is Turing-complete	19
5.1.2	Education Website showing Mobile Automata	19
5.1.3	Python Notebook to convert Turing Machine to Mobile Automata	19
5.2	Evaluation	19
5.3	Conclusions	20

5.4	Ideas for future work	20
5.4.1	Mobile Automata Busy Beaver	20
5.4.2	Website optimisation	20
5.4.3	Mobile Automata to Turing Machine emulation	21
5.4.4	Languages Recognised by Mobile Automata	21
References		22
Appendices		24
A Self-appraisal		24
A.1	Critical self-evaluation	24
A.2	Personal reflection and lessons learned	24
A.2.1	Time allocation	24
A.2.2	Objectives	24
A.2.3	Code implementation	24
A.2.4	Communication	25
A.3	Legal, social, ethical and professional issues	25
A.3.1	Legal issues	25
A.3.2	Social issues	25
A.3.3	Ethical issues	25
A.3.4	Professional issues	25
B External Material		26
B.0.1	Libraries and Tools	26
B.0.2	External Images	26
B.1	Notation	26
B.2	Definitions	27
B.3	Proof that every MA can be represented by a TM	27
B.4	Proof that TM can be represented with an MA	29

Chapter 1

Introduction

1.1 Outline

This report bridges the gap between academic research and hobbyist innovation. It intends to introduce and formalise a model of computing known as the Mobile Automata that is able to recognize recursively enumerable (RE) languages, thus having equivalent capability as the Turing machine model of computing. Overall, the report hopes to inspire the reader into experimenting with the Mobile Automata by providing a formal definition, a website to introduce it and its relevance in an academic context.

Initially, the report discusses background research divided into two sections. The first section concerns academic literature concerning formal languages and automata theory and places the Turing Machine in this world. The next section proceeds to introduce the Mobile Automata, where it was first conceptualised in Stephen Wolfram's book '*A New Kind of Science*' [21] and how it was adopted by the community forum named the '*Wolfram Community*' [23].

In the following chapter, a formalism of the Mobile Automata is created and explained, highlighting how this is meant to be compatible with the Turing Machines formal definition. Simplifications to the model are also justified and accompanied by informative diagrams.

Following this, a website which is designed to introduce the operation of the Mobile Automata is presented. Explanations are given regarding the design process, code framework used and limitations. This section also states inspirations and ideal use cases.

To conclude, the report discusses the implications of the Mobile Automata and future work than can be done. In the appendix, the proof that establishes equality of the Turing Machine and the Mobile Automata is given using formal language theory and is discussed as a result.

1.2 Ethical considerations

This project has considered ethical perspectives identified by the Association for Computing Machinery (ACM) Code of Ethics and Professional Conduct [2]. This framework outlines fundamental principles to safeguard harm avoidance, honesty, fairness, respect for new ideas and trustworthiness in both individual and organisational computer science practice and research. As this project is primarily theoretical in nature and does not involve using sensitive data or include the development of software that has a direct real-world application, the ethical considerations are low risk and there are no immediate considerations related to privacy, consent, confidentiality or information/data misuse. This report draws on previously published work and care has been taken to ensure academic integrity with sources acknowledged according to referenced protocol.

Chapter 2

Background Research

The research for this report is segmented into two distinct types of sources. The first that will be presented is from more academic sources, and the latter is derived from more hobbyist sources.

2.1 Introduction to the Theory of Computation

The *theory of computation* is a branch of computer science that investigates what problems can be solved using various models of computation. When algorithmic solutions exist for a problem within a given model, it is possible to evaluate solutions efficiency. If no solution is found, an attempt can be made to constrain or reframe the problem until it becomes solvable [20].

A central area of this field is *automata theory*, which studies different models of computation, known as *automata*. Each type of automaton is analysed to understand the kinds of computational problems it can or cannot solve.

2.1.1 Understanding Problems Through Formal Languages

A common computational task is to determine whether an object has a particular property. For example, identifying which natural numbers are prime requires first establishing a representation for the natural numbers. Using digits like ‘1, 2, 3, ..., 0’, any natural number can be represented as a string of symbols (e.g., "92", "1001"). The set of all such strings forms a *language*. The goal becomes identifying which strings in this language correspond to prime numbers. A *formal language* is a set of strings (sequences of symbols) governed by precise rules, collectively known as a *formal grammar*.

To illustrate, consider checking the natural numbers to determine whether they’re even. The rule to confirm if a number is even can be checked if the final symbol is in the set ‘0, 2, 4, 6, 8’. This would mean that "102" would be accepted, therefore complying with the language governed by this rule. However "101" would not be in the language because the rule does not permit it.

2.1.2 Automata and the Chomsky Hierarchy

Different models of computation are categorized based on the classes of formal languages they can recognize. More powerful automata can recognize more complex languages. This classification, introduced by Noam Chomsky, is known as the **Chomsky hierarchy** [19].

From least to most powerful, the hierarchy includes:

- Combinational circuits
- Finite-state machines
- Pushdown automata
- Turing machines

Each higher level can simulate all lower ones. For example, the *Turing machine* subsumes the capabilities of both finite-state machines and pushdown automata.

Returning to the prime number example: determining whether a number is prime requires a Turing machine, as the set of prime numbers forms a *recursive language*. Less powerful automata, such as pushdown automata, lack the expressive power to solve this problem. [10]

2.1.3 The Turing Machine and Its Limitations

The *Turing machine* (TM) is a model of computation capable of recognizing **recursively enumerable languages**, the most general class of formal languages currently known. Defined with an infinite memory tape, the Turing machine is not physically realizable and differs significantly from modern computer architectures. Despite its power, the Turing machine is not omnipotent. Alan Turing famously proved the *Halting Problem* to be **undecidable**: no Turing machine can universally determine whether another machine will halt on a given input.

In his seminal 1936 paper, Turing also proved that no Turing machine can solve the *Entscheidungsproblem*, which asks whether a mechanical method exists to determine the truth of any statement in first-order logic. His proof relied on the fact that solving the Entscheidungsproblem would require a solution to the Halting Problem, leading to a contradiction [16].

These findings, alongside Gödel's Incompleteness Theorems [9], expose fundamental limitations in computation and formal logic.

2.1.4 Other Models of Computation

The Turing machine is not the only model in its class. *Lambda calculus*, developed by Alonzo Church (Turing's doctoral advisor), is computationally equivalent to the Turing machine. Each can simulate the other, placing them in the same class. [17]

Other models shown to be *Turing-complete* include:

- Rule 110 (an elementary cellular automaton) [5]
- Conway's Game of Life [14]
- Folding origami (proved by emulating rule 110) [12]

These models can all recognize recursively enumerable languages, emphasizing that the Turing machine is not unique in this respect. Nonetheless, due to its simplicity and ubiquity, the Turing machine is often used as the standard when evaluating the power of other computational models.

2.1.5 The Church–Turing Conjecture

The *Church–Turing conjecture* posits that *any function computable through an effective, mechanical process can be computed by a Turing machine*. This suggests that Turing machines (or equivalent models) represent the theoretical upper bound of computation. [16]

If true, it is remarkable that humans have managed to conceptualize the full extent of computability. While the conjecture remains unproven, much like the unresolved $P = NP$ problem, it is broadly accepted within the scientific community. Still, it leaves open the possibility that even more powerful computational models could be discovered in the future.

2.1.6 Textbook Definition of the Turing Machines

Now that the context of the Turing Machine is known and what its capable of, here is its formal definition. Note that this definition varies, so a specific definition has been taken from the textbook '*Introduction to Automata Theory, Languages, and Computation*' [11].

The Turing Machine is a 7 tuple of the following:

$$TM = (Q, \Sigma, \Gamma, \delta, q_0, b, F)$$

Each component have the following meanings:

- Q : The finite set of states.
- Σ : The finite set of input symbols.
- Γ : The complete set of tape symbols; Σ is always a subset of Γ .
- δ : The transition function. The arguments of δ are a state q and a tape symbol x . The value of $\delta(x, q)$, if defined, is a triple (y, p, d) , where:
 1. p is the next state, in Q .
 2. y is the symbol, in Γ , written in the cell being scanned, replacing whatever symbol was there.
 3. d is a direction, either L or R , standing for "left" or "right," respectively, and telling us the direction in which the head moves.
- q_0 : The start state, a member of Q , in which the finite control is found initially.
- b : The blank symbol. This symbol is in Γ but not in Σ ; i.e., it is not an input symbol. The blank appears initially in all but the finite number of initial cells that hold input symbols.
- F : The set of final states. Final states can either be accepting or rejecting. On termination, this determines whether we accept the input or reject the input.

2.2 Mobile Automata

2.2.1 A New Kind of Science

Stephen Wolfram's *A New Kind of Science* [21] introduces the concept of *Mobile Automata*, although it occupies only a small portion of the book. Wolfram argues that simple computable rules can lead to complex behaviour, potentially offering explanations for natural phenomena. He advocates for a scientific approach to exploring these rules, referring to the entire space of such rules as the *ruliad*, and the study of it as *ruliology*. Rather than treating computation purely as a theoretical domain, the book emphasizes the value of experimental investigation, mirroring real-world scientific experimentation, where interesting results emerge through exploration.

Because of its relevance to Mobile Automata, some of Wolfram's ideas will be briefly discussed here. In the context of the Turing Machine and the formal definition of Mobile Automata presented later in this report, what *A New Kind of Science* refers to as *rules* corresponds to the transition functions available to an automaton.

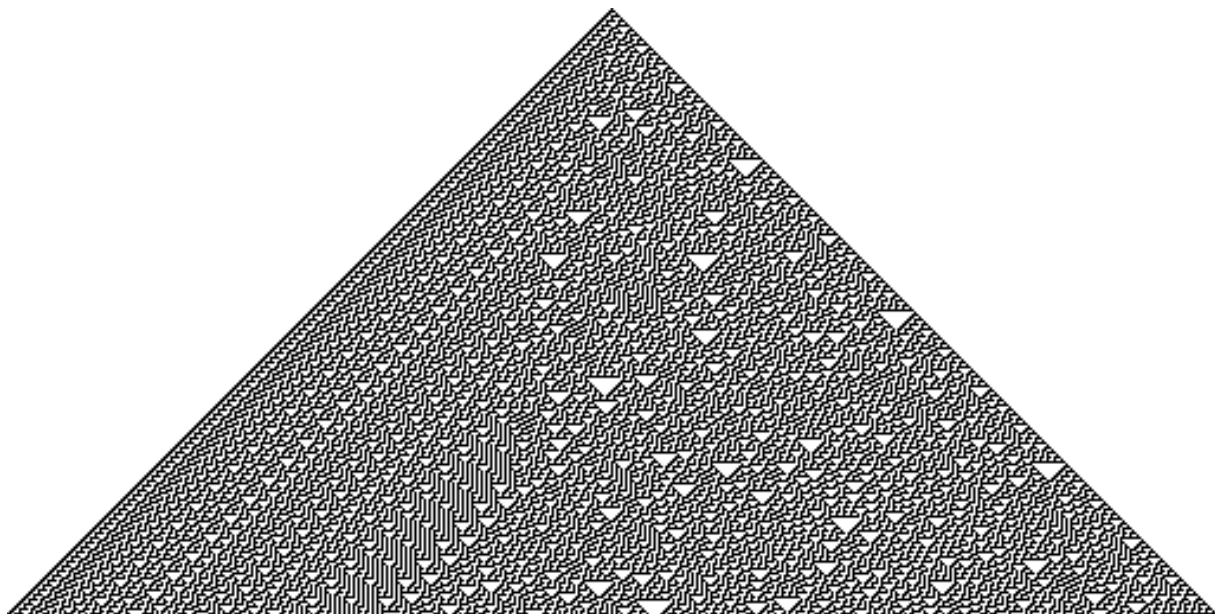


Figure 2.1: Rule 30

Cellular Automata

Cellular automata are the primary example Wolfram uses to demonstrate how simple rules can generate complexity. A cellular automaton typically consists of a row of cells, where each cell updates its state based on a fixed rule applied to itself and its neighbouring cells. Repeated application of this rule across generations produces successive rows, revealing patterns that emerge over time.

One recurring cellular automaton featured in *A New Kind of Science* is known as "Rule 30." Among the 256 possible variants of the elementary cellular automata Wolfram studies, Rule 30 stands out as the first in his enumeration to exhibit what he describes as *chaotic* behaviour. In this context, "chaotic" does not carry its formal mathematical meaning; rather, it refers to the appearance of noisy, visually random patterns in the automaton's output. An illustration of this automaton is shown in Figure 2.1.

Cellular automata have developed a small cult following that embraces Wolfram's perspective on their scientific significance. Enthusiasts have worked to classify different types of cellular automata and explore potential connections to real-world phenomena. One example Wolfram gives is the *Conus textile* shell, in which he proposes that as the cone grows, its pigmentation follows a computation similar to Rule 30 (*A New Kind of Science*, p. 423), as seen in Figure 2.2.

Figures in *A New Kind of Science*

A New Kind of Science features distinctive diagrams that illustrate the behaviour of various automata. Figures 2.3 and 2.1 provide examples, depicting a Mobile Automaton and the cellular automaton Rule 30, respectively. Although these represent different types of automata, they can both be visualized using the same style of diagram. Figure 2.4 further presents the rule that governs the behaviour of the Mobile Automaton. Together, these diagrams help visually convey the mechanisms by which rule-based systems evolve over time.

Each row in these diagrams, though not explicitly separated by lines, represents the state of



Figure 2.2: Rule 30 seen on a conus textile shell

the array at a given time step. The first row, at the top, of the diagram represents the input to the machine. Consecutive rows shows the result of applying the automaton's governing rule to the previous row above it. Each element is shown as either black or white, indicating the value stored in the array at that position. For these diagrams, the symbol 0 is represented by the colour white, and the symbol 1 is represented by the colour black. However, this can be expanded upon by including more colours to represent more symbols. For cellular automata, entire rows are updated simultaneously with each rule application. In contrast, Mobile Automata update only a local region around a moving point, represented by a black filled circle. This distinction will become clearer once the operational details of Mobile Automata are discussed.

Origin of the Mobile Automaton The Mobile Automaton is first introduced in *A New Kind of Science* (p. 71) as a "cellular automaton that updates locally." In traditional cellular automata, the rule is applied globally to the entire array, resulting in a full-row update at each step. In Mobile Automata however, the update is local, only affecting a small region. This localized behaviour is evident in the rule diagram (Figure 2.4), where the update occurs around the position marked by the black circle. The Mobile Automaton was introduced to explore whether the complex behaviours seen in cellular automata arise from global rule application, or whether similar behaviours can emerge from purely local updates.

2.2.2 Research on Mobile Automata in the Wolfram Community

Hobbyist research related to *A New Kind of Science* can be found on the online forum known as the **Wolfram Community**. This platform allows users to upload Wolfram Notebooks, similar in structure and functionality to Jupyter notebooks, which can include runnable code directly in the browser. This makes it an ideal medium for collaborative exploration, as shared code can be freely viewed, edited, and extended by others. Studying the work of others on Mobile Automata is a valuable way to uncover new properties and behaviours. As this kind of exploration falls under what Wolfram terms *ruliology*, most discussions and notebooks refer to different automata in terms of their defining *rules*.

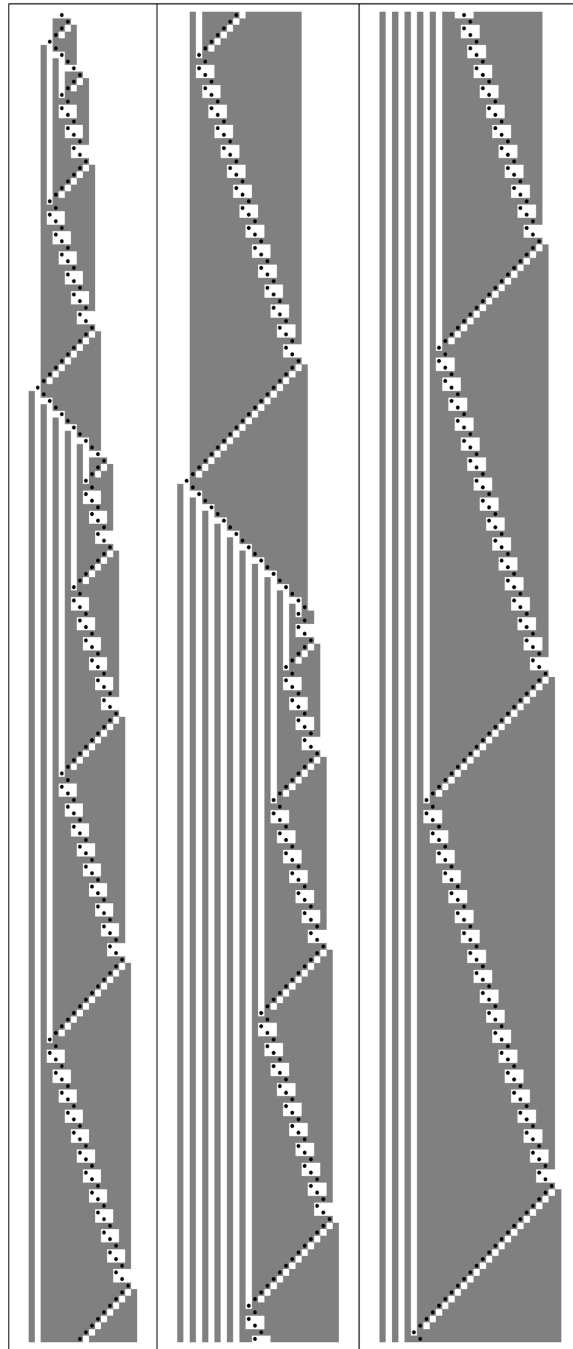


Figure 2.3: Mobile Automata in operation

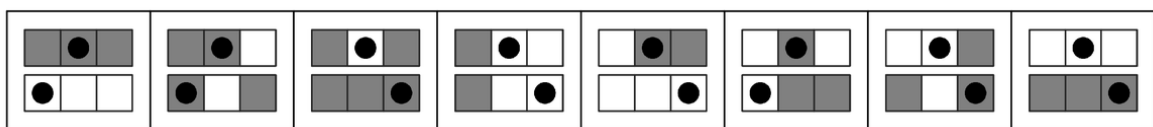


Figure 2.4: Mobile Automata Rules

Reversible Mobile Automata The notebook *Reversibility in Mobile Automata* [8] explores the concept of *reversibility* in Mobile Automata, outlining the conditions under which an automaton exhibits this property. If an automaton is reversible, it means that every configuration in its evolution has a unique predecessor—allowing the computation to be run backward without ambiguity.

For example, the logical operation **NOT** is reversible, since applying it twice returns the original input:

$$\mathbf{NOT}(\mathbf{NOT}(a)) = a$$

In contrast, the **AND** operation is not reversible:

$$\mathbf{AND}(0, 1) = 0 = \mathbf{AND}(1, 0)$$

Despite yielding the same result, the inputs differ, meaning that from the output alone, the original inputs cannot be uniquely determined. For the mathematically inclined, this relates to whether a function is *injective*, although the terminology in the notebook differs.

The notebook is inspired by Wolfram’s broader view that computation is deeply connected to the fundamental laws of physics. According to the second law of thermodynamics, entropy must always increase over time. If physical laws are time-symmetric, then this increase in entropy occurs regardless of whether time moves forward or backward. This can then be seen as an irreversible operation, even if we consider moving backwards in time, reinforcing the importance of studying reversibility in computational systems [22]. There may be a connection in finding an irreversible computation that corresponds to the irreversibility of the second law of thermodynamics.

Characterization of Mobile Automata

The work done by Lap Sum Chan in this notebook [3] is to categorize how Mobile Automata behave differently depending on their rules. He studies the Mobile Automata the way they are presented in *A New Kind of Science* meaning there are 65536 rules to be categorized. Due to hardware constraints, all of these rules cannot be considered at once and a random subset of 1000 rules are taken to be examined. Each of these 1000 rules are assigned to a Mobile Automata and given the same initial configuration and run for 60 steps.

These different computations plotted with a clustering algorithm and labelled. The notebook highlights the biggest difference between each computation is the direction of the head movement, and thus the resulting output. Some rules lead the Mobile Automata to enter a cycle or habitually editing the same region, while others present movement across the whole of the input and are far less predictable. To conclude, the notebook states more should be done to understand what makes some rules lead to unpredictable movement by the Mobile Automata and perhaps how we can predict if a Mobile Automata will become cyclic given its rules.

Ruliology of Network Mobile Automata

This is an example of a Mobile Automata conceptualised in a different medium. In this notebook, Joel Choi [4] implements Mobile Automata into graphs, where the rules take sub-graphs around the head of the Network Mobile Automata as input as replaces a new sub-graph in their place.

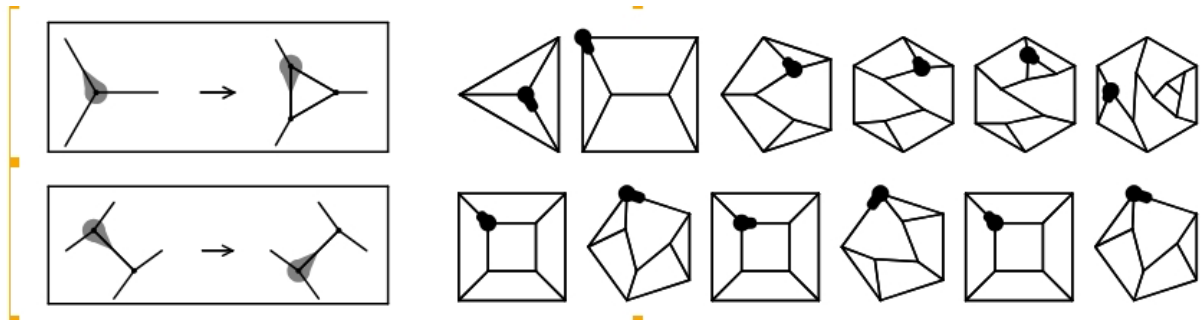


Figure 2.5: Network Mobile Automata

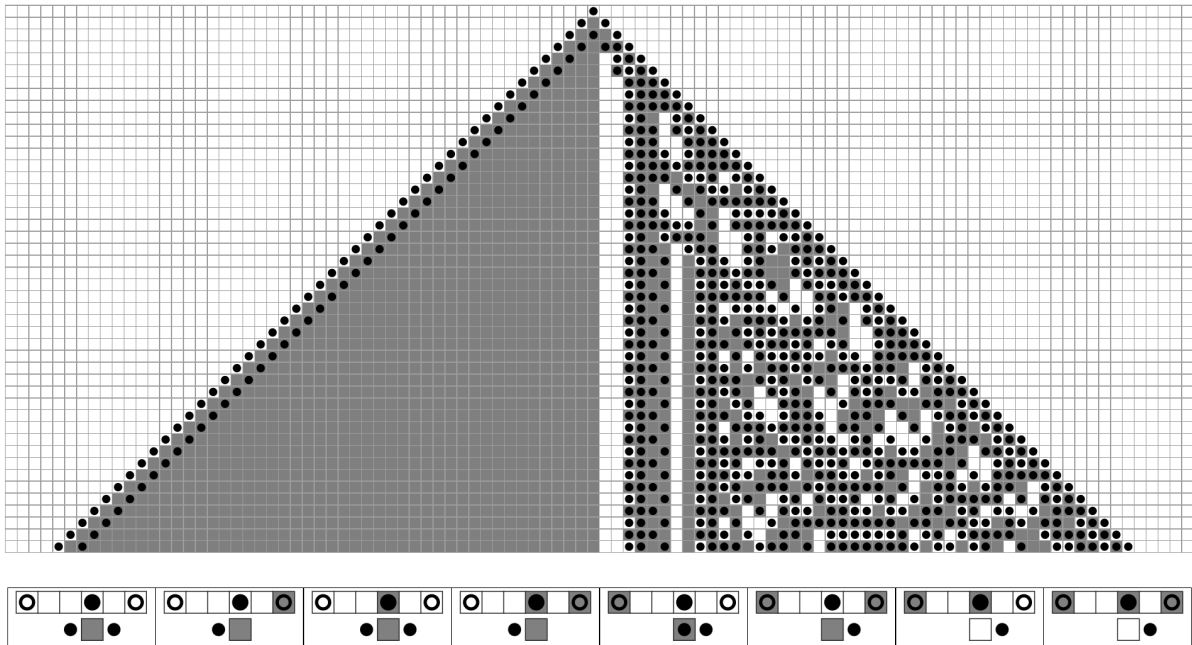


Figure 2.6: Mobile Automata with non-local rules given at bottom of figure, initialised on an 'empty' (all zero) input.

The inspiration for this was taken from *A New Kind of Science* (Chapter 9, Section 12). Refer to Figure 2.5 for an example of a Network Mobile Automata in action. Each row of the figure shows a separate Network Mobile Automata, with the rule for such automata listed on the left hand side. Notice the rule includes the orientation of the automata. The input is the left graph on the diagram, and each consecutive step right is an application of the Network Mobile Automata.

The notebook compares and combines Network Mobile Automata with hyper graph rewriting rules, which is a reoccurring concept in the *Wolfram Physics Project*, however this relies beyond the grasp of this report. *Ruliology of Network Mobile Automata* because I find it to be another example of variations and creativity embraced upon the concept of a Mobile Automata.

Mobile Automata with non-local rules

This is the most outrageous example I have of an adapted Mobile Automata given by Akshaj Devireddy [6]. Referring to Figure 2.6, the rules of the variant of Mobile Automata are drastically different to what we have seen in Figure 2.4. What remains similar, is that the Mobile Automata only alters one cell. However before the Mobile Automata was limited in moving once right

or once left, the Mobile Automata given in *Mobile Automata with non-local rules* are able to duplicate which are seen in figure 2.6. Remembering the head of the mobile automata is the black filled circle, outputs containing two filled black circles mean there are now 2 non-local Mobile Automata produced. Cells with a Mobile Automata over them (a black filled circle) are known as active cells. In this variant of Mobile Automata, still 3 input cells are being read as in conventional Mobile Automata, though as the title implies this is happening 'non-locally' and is given by cells with a hollow circle over them in the input of the Mobile Automata.

The notebook experiments with parameters on these 'non-local Mobile Automata', experimenting with the range that the input spans and seeing what effects this has. This notebook is mentioned again for its innovation on Mobile Automata. It was created by a secondary school student exploring a previously unexplored area. From the acknowledgements, it strengthened the students ability to read and write code as well as peaking their interest.

Chapter 3

Formalising Mobile Automata

3.1 Example of a Mobile Automata

Before an example is given, it must be made clear that this is an adaptation of the Mobile Automata from Wolfram's initial description and that studied on the Wolfram Community. It has been adapted to be more versatile and compatible with the modern textbook *"Introduction to Automata Theory, Languages and Computation"* [11].

In figure 3.2, 3.1 there are two example of transitions a mobile automata can make. In the figures, the grey cells correspond to a symbol 1 and white to the symbol 0, as seen in *"A New Kind of Science"*. The black dot marks the head of the Mobile Automata, where it may read a word left to right. In figure 3.1, the image explains if a Mobile Automata reads the word 001, it will write 100 in its place and move to the right. In figure 3.2, the image explains if a Mobile Automata reads 000, it will move left and write 101 in its place. In the formal definition that will be given later in the report, this would be read as $\delta(001) = (100, R)$ and $\delta(000) = (101, L)$.

Figure 3.3 shows the Mobile Automata containing the rules as seen in 3.2, 3.1 being executed on a input of all zero.

3.2 Formal definition of a Mobile Automata

We create a definition of the Mobile Automata to be the 6-tuple:

$$MA = (n, \Sigma, \Gamma, \delta, b, F)$$

where the components have the following meanings:

- n : The length of words we read off the tape at a step in computation. For the purposes of this report it is odd and greater than 1.
- Σ : The finite set of input symbols.
- Γ : The complete set of tape symbols; Σ is always a subset of Γ .

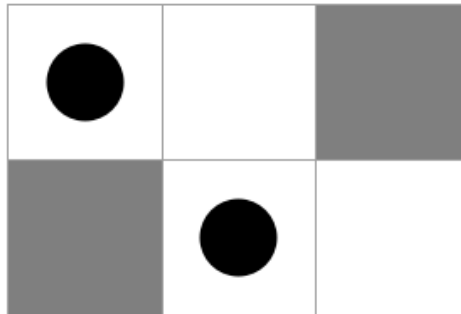


Figure 3.1: Right moving Mobile Automata transition for $n=3$ on a binary input

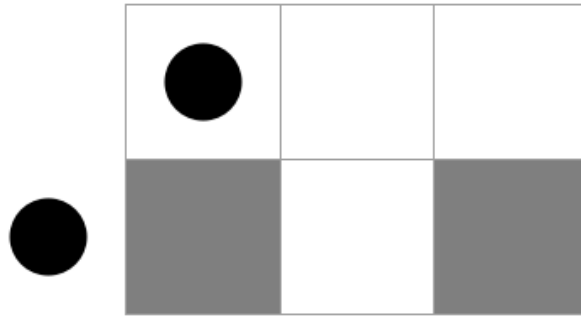


Figure 3.2: Left moving transition rule for Mobile Automata where $n=3$ on a binary input

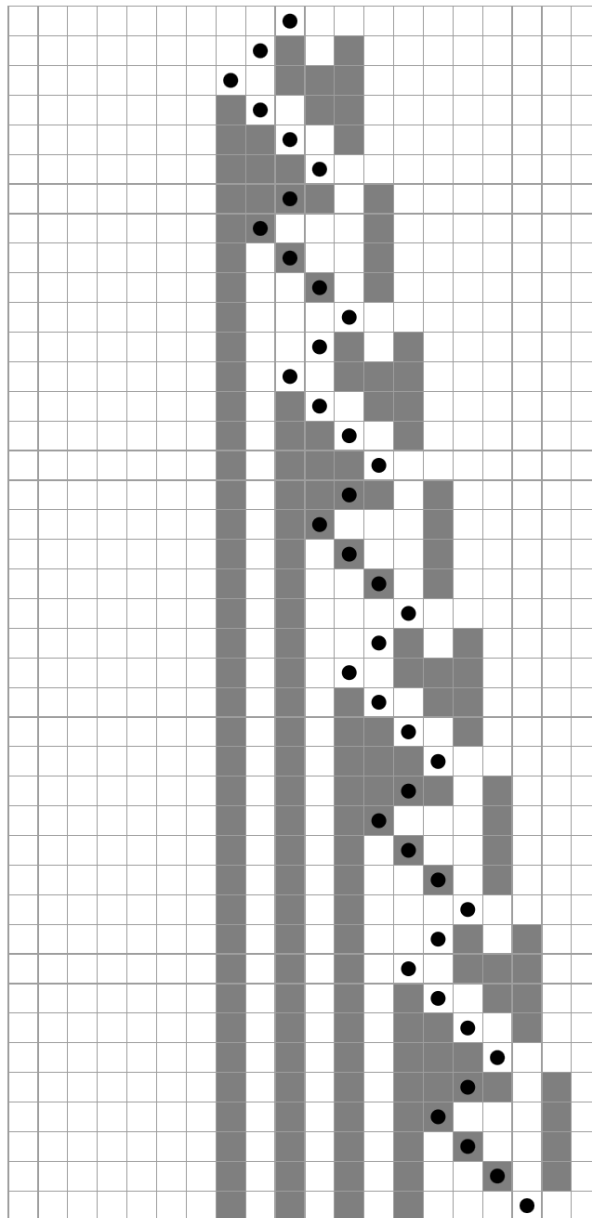


Figure 3.3: Example of Mobile Automata with the transition function as shown above

- δ : The transition function. The arguments of δ is a word $x \in \Gamma^n$. The value of $\delta(x)$, if defined, is a triple (y, d) , where:
 1. $y \in \Gamma^n$ is the word written onto the word x being scanned.
 2. d is a direction, either L or R , standing for "left" or "right," respectively, and telling us the direction in which the head moves.
- b : The blank symbol. This symbol is in Γ but not in Σ ; i.e., it is not an input symbol. The blank appears initially in all but the finite number of initial cells that hold input symbols.
- F : The set of words $F \subseteq \Gamma^n$ that are not in the domain of δ and have no mapping, there for being the set of words for which the MA terminates. This can be partitioned into accepting words and rejecting words.

3.3 Justification of design choices

3.3.1 Similarity to Truing Machine

Following intuitively from *"Introduction to Automata Theory, Languages, and Computation"* [11], the definition was to appeal intuitive to students who have studied the Turing Machines definition. Symbols such as $\Gamma, \delta, \Sigma, b, F$ are kept the same and have similar purposes as in the definition of the Turing Machine.

3.3.2 Left-Right Reading

As mentioned earlier, this Mobile Automata isn't the same as seen in *'A New Kind of Science'*. One major difference is that the input word is read left to write from the head of the Mobile Automata, not radially around it. This allows the Mobile Automata to take words of even length, where as radially would always necessitate words of odd lengths.

3.3.3 Terminating machine

Mobile Automata as they are known in the Wolfram community don't have a concept of terminating, which is a requirement for Mobile Automata to be compatible with formal language theory. As such, the definition includes terminating words F which can be partitioned into accepting terminating words and rejecting terminating words. This is used in the proof that Mobile Automata are Turing-complete due to them both having capability to accept the same language.

It can be noted that this is implied to be the set of words that are not in the domain of δ , as any word that doesn't have a mapping would mean the Mobile Automata has no way of continuing.

Chapter 4

Implementation

4.1 Mobile Automata Visualization Website

4.1.1 Motivation

This report serves as an introduction to Mobile Automata and likewise a software implementation should serve the same purpose. An optimal way to achieve this was to create a website that would display a Mobile Automata in operation to provide intuition to its mechanism.

4.1.2 Design and Implementation

Programming Languages used

Since Mobile Automata are simple to implement, the website was implemented with plain JavaScript and HTML. This as websites are accessible and lightweight so they can be viewed on many different machines, which is in line with the intended use case. In comparison, using a programming language such as **C** would enable Mobile Automata to be run more efficiently but would be less accessible. The intention is that the website serves as an introduction, if users are inclined to continue working on the Mobile Automata, they can seek more efficient methods independently. This is stated clearly to the user so that they are aware of the limitations.

For HTML and JavaScript libraries, jQuery [13] was used for extracting DOM elements efficiently and Tailwind CSS [18] was used for simple CSS formatting. Aside from these, there were no other libraries used as Mobile Automata are easy to implement.

Inspiration

Inspiration for the website was drawn from turingmachine.io [1]. The site is structured around three main components:

- State Diagram
- Tape Visualization
- Instruction Editor

The State Diagram is not essential for Mobile Automata, so the primary focus is on the Tape Visualization and Instruction Editor.

The Tape Visualization aligns with the formal definition by displaying symbols and transitions clearly. Compared to the Mobile Automata figures discussed earlier 2, the Turing Machine visualization explicitly draws symbols within cells, while Mobile Automata figures typically use coloured cells. Another difference is in representing the machine's head: the Turing Machine marks it with an outlined cell, whereas Mobile Automata use a solid black circle. Additionally, only one tape instance is shown at a time, with an animation illustrating each application of the

Mobile Automata Simulator

Input Tape

Tape Contents:

Head Position:

Mobile Automaton Definition

Transition Function (δ):

000,111,R
010,101,L
10,111,R
111,000,R
012,111,R

Accepting Words:

Blank Symbol:

Input Size (n):

Input Alphabet (Σ):

Working Alphabet (Γ):

Terminating Words:

Figure 4.1: The input to the Mobile Automata Simulator website. Grey fields are uneditable as they may be derived from the transition function.

Tape Display

-	-	-	-	-	-	-	-	-	1	0	0	0	1	0	1	0	1	0	1	-	-	-	-
											1	1	1										

Machine has terminated and **REJECTS!**

Figure 4.2: Tape display with termination notification. In this example, the machine has terminated on the word 001 and the last transition converted 111 into 000 moving right.

transition function. The style and clarity of this Tape Visualization were chosen as a model, as they offer a seamless and intuitive view of the Turing Machine’s operation.

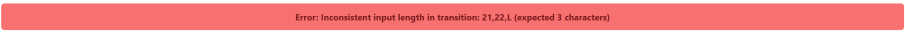
The Instruction Editor is also a valuable feature, allowing Turing Machine instructions to be shared easily using YAML. Users who create Turing Machines for specific tasks, such as the **BusyBeavers** included among the saved YAML examples, can exchange their designs conveniently. This functionality is particularly beneficial for newcomers experimenting with transition functions, who might create humorous examples or unique machines and wish to share their work with others.

4.1.3 Deployed Website

Workflow

The workflow for this website follows a two step procedure:

1. Enter desired transition function and input. The website has been designed to automatically detect input symbols and tape symbols. Since these symbols can be inferred from the machine’s input and transition function, removing the need for manual entry simplifies the user experience. Figure 4.1 illustrates the input interface. On the left, users can specify the initial tape contents and the head position. On the right, users can define the transition function, blank symbol, and accepting words. Fields that are greyed out are automatically derived and cannot be edited; they are shown to inform users of the values without requiring manual input.
2. Execute Mobile Automata on given input. This is done with the usage the buttons:
 - ‘step’ - Performs one execution of the Mobile Automata.
 - ‘steps’ - Performs 10 executions of the Mobile Automata.



Error: Inconsistent input length in transition: 21,22,1 (expected 3 characters)

Figure 4.3: Error message on incorrect input

- 'reset' - Resets the Mobile Automata to the input.

During execution, an animation is played to show the user which word has been replaced by another. As shown in figure 4.2, the last word read was "111", which was substituted with "000", followed by a move to the right. This animation makes the process more intuitive when using the website. Performing a 'step' or 'steps' continues the execution until the Mobile Automata terminates, at which point a green message indicates the input was accepted, or a red message indicates it was rejected.

Input Sanitization

The website includes input sanitization for the transition function. Because Mobile Automata require the transition function to input and output words of the same length, the code enforces this constraint. Figure 4.3 shows an error message displayed in the same area that indicates whether the Mobile Automata has terminated. These messages provide clear guidance on how to fix mistakes, making corrections straightforward.

Limitations

Since the website depends on HTML animations and transitions, it encounters issues if the tab is unfocused while the Mobile Automata is running. When a tab loses focus, CSS transitions pause, but the Mobile Automata's internal functions continue, causing them to fall out of sync. This could be resolved by either refreshing the tape display upon refocusing or by switching to a different animation method, such as SVG. However, due to time constraints, this improvement was not implemented.

Additionally, as discussed in the background research, Mobile Automata can be extended to support features like multiple heads by duplicating the head. This was not implemented, and the site follows the formal definition of Mobile Automata presented in this report.

4.2 Mobile Automata Proof

4.2.1 Proof Implementation

A major achievement of this report is demonstrating that Mobile Automata can recognize recursively enumerable languages, including constructions for converting Mobile Automata to Turing Machines and vice versa. The full proof is presented in the appendix. This section aims to outline the methodology used to establish the proof and explain its compatibility with the textbook *Introduction to Automata Theory, Languages, and Computation* [11].

The proof approach was inspired by the example 'Equivalence of One-tape and Multitape Turing Machines' (chapter 8, section 4) from the textbook. That proof shows that any language recognized by a Multitape Turing Machine can also be recognized by a standard single-tape Turing Machine. It does so by constructing a single-tape machine that simulates the multitape



Figure 4.4: Busy Beaver 5 implemented with a Mobile Automata

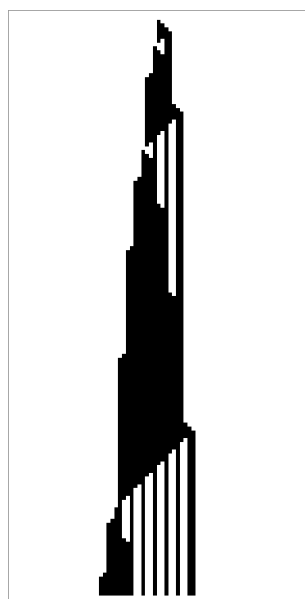


Figure 4.5: Busy Beaver 5 implemented with a Turing Machine, states have not been displayed with this figure

machine and demonstrating that both machines accept or reject input under identical conditions. Although the textbook does not provide an explicit construction, it includes a helpful diagram to illustrate the idea.

In contrast, the proof in this report introduces new terminology, such as describing one automaton as "representing" another, with definitions chosen for clarity and intuition. Unlike the textbook, this report provides explicit constructions of the machines involved and uses a more formal approach to establishing the result.

4.2.2 Proof Validation with Busy Beaver

To test the effectiveness of the proof that establishes the equivalence between Turing Machines and Mobile Automata, a Jupyter notebook was created to verify this result. Figure 4.4 shows the first few steps of the Busy-Beaver 5 emulated by a Mobile Automata using the proof found in the appendix. Figure 4.5 shows the original Busy-Beaver 5 as it would be on a Turing Machine.

The two figures are nearly identical, differing primarily in their colour schemes. This difference arises because the Mobile Automata simulates the Turing Machine using a larger alphabet, which requires a broader range of colours to represent the additional symbols. Another key detail in figure 4.4 is the presence of red highlights along the path of the Mobile Automata's head. These highlights indicate how state information is encoded in the movement of the head, influencing the symbols it writes. Viewing this figure in conjunction with the formal proof may provide additional insight into its structure and behaviour.

Chapter 5

Results and Evaluation

5.1 Results

5.1.1 The Mobile Automata is Turing-complete

A major outcome of this project is the successful formal proof that Mobile Automata are Turing-complete. By constructing transformations in both directions, from Turing Machines to Mobile Automata, and vice versa. The report confirms that Mobile Automata and Turing Machines can recognize the same set of recursively enumerable languages. This equivalence was rigorously proven using formal language theory and verified through a simulation of the Busy Beaver 5 Turing Machine as a Mobile Automaton.

5.1.2 Education Website showing Mobile Automata

A web-based visualisation tool was developed to showcase how Mobile Automata function. This interactive tool allows users to define transition rules and input values, and to observe step-by-step animations of the automaton in action. It was specifically designed to be lightweight, accessible, and educational. The website automatically sanitizes inputs and provides meaningful feedback to help users understand the structure and behaviour of their machines.

5.1.3 Python Notebook to convert Turing Machine to Mobile Automata

To verify the theoretical equivalence in practice, a Python Jupyter notebook was created that converts Turing Machines into their Mobile Automaton equivalents. This notebook was successfully used to simulate the 5-state Busy Beaver, demonstrating the accuracy of the conversion and offering a practical reference implementation of the proof.

5.2 Evaluation

The project achieved its core objectives: defining Mobile Automata in a way that aligns with formal language theory, proving their equivalence to Turing Machines, and producing educational tools to make this concept more accessible.

The **formal equivalence proof** was a key strength. Unlike many informal or intuition-based comparisons, this project provided explicit constructions for representing Mobile Automata with Turing Machines and vice versa. The proof maintained alignment with standard academic definitions, particularly those from the widely used textbook *Introduction to Automata Theory, Languages, and Computation*. This helped ensure both rigour and compatibility with conventional theory.

The **educational website** proved effective in helping users intuitively understand the mechanics of Mobile Automata. It handled user input gracefully and visually animated the transi-

tion function, which is typically abstract and difficult to conceptualise. The website’s limitations were noted, but these did not affect its informative value for most users.

The Jupyter notebook served not only as a validation tool for the theoretical proof but also as a bridge between formalism and practice. Successfully simulating the 5-state Busy Beaver demonstrated that the Mobile Automata can perform complex, real-world computations.

5.3 Conclusions

This project formalised the concept of Mobile Automata and demonstrated their computational equivalence to Turing Machines. Through rigorous theoretical work and practical implementation, it established that Mobile Automata can accept the same class of languages as Turing Machines.

The formal definition of Mobile Automata developed here introduced key innovations, such as localised transitions with head movement and a termination mechanism based on undefined transitions. These adjustments allowed the model to integrate with formal language theory in a way that Wolfram’s original version did not.

By implementing an educational website and a Python-based conversion tool, the project bridged the gap between abstract theory and concrete understanding. These tools help others learn about Mobile Automata through experimentation and visualisation.

Overall, the project showed that Mobile Automata are more than a theoretical curiosity; they are a legitimate model of computation worthy of further study. That said, the project could be improved in several areas, including performance optimisations in the website, better handling of non-halting behaviours, and exploration of additional Mobile Automata variants, such as those with multiple heads or non-local rules.

5.4 Ideas for future work

5.4.1 Mobile Automata Busy Beaver

An intriguing direction for future exploration is adapting the Busy Beaver challenge to Mobile Automata. Just as the Busy Beaver function seeks the Turing Machine with n states that runs the longest before halting, a corresponding challenge could be defined for Mobile Automata with fixed transition rules and word length n . This would open a new line of theoretical inquiry, providing insight into the time complexity and halting behaviour of various Mobile Automata configurations.

Developing a Mobile Automata Busy Beaver function could also lead to deeper comparisons between the growth rates and expressiveness of Mobile Automata and standard Turing Machines. It may help uncover structural differences in how each model handle time until termination.

5.4.2 Website optimisation

Optimising the educational website’s rendering logic, possibly replacing CSS animations with SVG or canvas-based rendering, could improve usability and accuracy. Furthermore, allowing the user to experiment with different kind of Mobile Automata such as those that were seen in background research may provide a intriguing experience to users.

5.4.3 Mobile Automata to Turing Machine emulation

As done in this report, there was a Jupyter notebook that converted the transition function of a Turing Machine into a Mobile Automata. However, the converse is still to be done, which is converting the transition function of a Mobile Automata into that of a Turing Machine. This would not be a daunting task and the proof provided in the appendix could be used to complete this task.

5.4.4 Languages Recognised by Mobile Automata

In formal language theory, there are commonly referred to examples of languages that can only be understood by different types of automata. For example, the language $L = \{0^n 1^n, n \in \mathbb{N}\}$ is not a regular language but a context free language. This means it cannot be recognised by a *Finite State Machine* but can however be recognised by a *Pushdown Automata*. Since Mobile Automata are able to read recursively enumerable languages, there must be a way for a Mobile Automata to recognise this language and has still not been done. L is an example given here but there is a variety of other languages that a Mobile Automata could be built to recognise.

Bibliography

- [1] aepsilon. Turing machine visualizer. <https://github.com/aeppsiion/turing-machine-viz>, 2025. Accessed: 2025-04-17.
- [2] Association for Computing Machinery. Acm code of ethics and professional conduct, 2018. Accessed: 2025-04-29.
- [3] L. S. Chan. Exploration and characterization of mobile automata. <https://community.wolfram.com/groups/-/m/t/2960211>, 2023. Wolfram Community, Staff Picks, 13 July 2023. Accessed: 2025-04-21.
- [4] J. Choi. The ruliology of network mobile automata. <https://community.wolfram.com/groups/-/m/t/2959221>, 2023. Wolfram Community, Staff Picks, 12 July 2023. Accessed: 2025-04-21.
- [5] M. Cook. A concrete view of rule 110 computation. *Electronic Proceedings in Theoretical Computer Science*, 1:31–55, June 2009.
- [6] A. Devireddy. [wsrp24] mobile automata with non-local rules. <https://community.wolfram.com/groups/-/m/t/3214519>, 2024. Wolfram Community, Evergreen Valley High School, Wolfram High School Summer Research Program.
- [7] P. S. Foundation. itertools — functions creating iterators for efficient looping. <https://docs.python.org/3/library/itertools.html>, 2024. Accessed: 2025-04-21.
- [8] A. French. Reversibility in mobile automata. <https://community.wolfram.com/groups/-/m/t/3210947>, 2024. Accessed: 2025-04-21.
- [9] K. Gödel. *On Formally Undecidable Propositions of Principia Mathematica and Related Systems*. Dover Publications, New York, 1962. Originally published in 1931 as *Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme*.
- [10] J. Hartmanis and H. B. Shank. On the recognition of primes by automata. *Journal of the ACM*, 15(3):382–389, 1968.
- [11] J. E. Hopcroft, R. Motwani, and J. D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Pearson, Boston, 3rd edition, 2007.
- [12] T. C. Hull and I. Zakharevich. Flat origami is turing complete, 2025.
- [13] T. jQuery Foundation. jquery: The write less, do more, javascript library. <https://jquery.com>, 2024. Accessed: 2025-04-21.
- [14] P. Rendell. *Turing Machine Universality of the Game of Life*. Emergence, Complexity and Computation. Springer Cham, 1 edition, 2016. Hardcover ISBN: 978-3-319-19841-5 (2015), Softcover ISBN: 978-3-319-37044-6 (2016).

- [15] L. Torvalds and G. Contributors. Git: Distributed version control system. <https://git-scm.com>, 2024. Accessed: 2025-04-21.
- [16] A. M. Turing. On computable numbers, with an application to the entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1):230–265, 1936. Correction in *Proceedings of the London Mathematical Society*, 1937, s2-43 (6): 544–546.
- [17] A. M. Turing. Computability and λ -definability. *The Journal of Symbolic Logic*, 2(4):153–163, 1937.
- [18] A. Wathan and Contributors. Tailwind css: A utility-first css framework. <https://tailwindcss.com>, 2024. Accessed: 2025-04-21.
- [19] Wikipedia contributors. Chomsky hierarchy — Wikipedia, the free encyclopedia, 2025. Accessed: 2025-02-25.
- [20] Wikipedia contributors. Theory of computation — Wikipedia, the free encyclopedia, 2025. Accessed: 2025-02-25.
- [21] S. Wolfram. *A New Kind of Science*. Wolfram Media, 2002.
- [22] S. Wolfram. Computational foundations for the second law of thermodynamics, 2023. Accessed: 2025-04-21.
- [23] Wolfram Community. Wolfram community, 2025. Accessed: 2025-04-29.
- [24] Wolfram Research, Inc. *Mathematica*. Wolfram Research, Inc., Champaign, Illinois, version 13.3 edition, 2023. Available at <https://www.wolfram.com/mathematica/>.

Appendix A

Self-appraisal

A.1 Critical self-evaluation

This project was an enlightening experience that deepened my understanding of formal proof writing, especially through the use of $\text{\LaTeX}$. Over the course of the report, I became much more comfortable with structuring mathematical arguments and presenting them in a clear, readable format. It also gave me the opportunity to engage with academic material more critically, such as reading, reviewing and discussing complex theoretical ideas in a meaningful way. One aspect I found particularly rewarding was connecting formal academic sources with more informal community-based work, such as discussions from the Wolfram Community. Bringing these two perspectives together helped me appreciate the broader relevance of the topic.

A major accomplishment of this project was showing how Mobile Automata can be viewed as more than just a curiosity. By proving that they are Turing-complete and can recognise recursively enumerable languages, I was able to demonstrate their significance in an academic context. This gives the model a stronger place within the existing body of computational theory, and I hope it encourages further exploration from others in the field.

Finally, my writing skills improved throughout the process of developing this report. As the project progressed, I became better at structuring arguments, presenting ideas clearly, and maintaining a consistent tone. I now feel more confident in both my technical and academic writing.

A.2 Personal reflection and lessons learned

A.2.1 Time allocation

I underestimated the time required to write the report and finalise all deliverables. Much of the writing was done toward the end of the project, which made it harder to evaluate the direction of the report. In the future, I intend to adopt a more incremental approach to documentation and balance it alongside development and research.

A.2.2 Objectives

The project would have benefited from a clearer definition of goals at the outset. While the scope was exploratory by design, having more concrete milestones would have helped track progress and reduce ambiguity.

A.2.3 Code implementation

Although the software tools met their functional goals, I encountered limitations with browser-based animation and in future projects I hope to learn SVG animation. Furthermore, though

raw HTML and JavaScript was efficient to deploy, it is difficult to scale and build upon. In future web-based projects I would make use of frameworks to help organise code.

A.2.4 Communication

While my supervisor was supportive, I could have communicated more consistently, especially when facing uncertainty. Sharing progress more regularly would likely have helped identify issues earlier and improve the final outcome.

A.3 Legal, social, ethical and professional issues

A.3.1 Legal issues

This project made use of *Wolfram Mathematica* [24], a proprietary software tool, under an academic license. All usage complied with the terms of the student subscription. Any visualisations or results generated using Mathematica are original and created for the purposes of this report, with proper acknowledgment of the tool's contribution.

A.3.2 Social issues

Even when drawing from informal sources such as community forums, it remains important to give proper credit to contributors. In this report, all examples and insights taken from community members have been appropriately acknowledged. Supporting a collaborative community requires more than passive use, it depends on recognition and constructive engagement. The Wolfram Community fosters this through contributor badges, but more meaningfully, it thrives when users provide thoughtful feedback, critiques, and extensions to each other's ideas.

A.3.3 Ethical issues

As the project is theoretical in nature and does not involve human subjects, sensitive data, or applications with real-world impact, it poses minimal ethical risk. Nonetheless, ethical standards in research were maintained by adhering to the ACM Code of Ethics and Professional Conduct [2], particularly in relation to honesty, fairness, and giving proper credit. All third-party content has been referenced appropriately to ensure academic integrity.

A.3.4 Professional issues

Given the lack of formal peer review during the project, it is important to subject the findings to critical scrutiny. Without external validation, results must be interpreted cautiously. This report encourages readers to examine and challenge its conclusions where appropriate.

Appendix B

External Material

B.0.1 Libraries and Tools

Javascript Libraries

- jQuery - Used to find elements in the HTML DOM conveniently [13]
- Tailwind CSS - Used to stylize HTML [18]

Python Libraries

- PIL - Used to convert 2D arrays to images
- numpy - Used for array manipulation
- itertools - Used for computing Cartesian product of sets [7]

Version control tools

- Git - Used to update code incrementally and track and label changes [15]

B.0.2 External Images

- Figure 2.3 ,2.4 - Mobile Automata plot and rule figure [21]
- Figure 2.1, 2.2 - Cellular Automata rule 30 plot and sea shell appearing to have natural likeness [21]

B.1 Notation

- A word x of length n is a tuple of n symbols: $x = x_0x_1...x_{n-1}$. The i^{th} component of x is x_i .
- Let $j_0, ..., j_{k-1}$ be an ordered list of k indices. $y_{j_0, ..., j_{k-1}} = y_{j_1}y_{j_2}...y_{j_k}$. That is, the word y arranged in order of the indices $j_1...j_k$.
- $y||\alpha$ is the symbol α concatenated onto the word y : $y||\alpha = y_0y_1...y_{n-1}\alpha$
- ϵ is the word of length 0.
- $\text{tape}(M, I, t)$ is the contents of the tape of a machine M executed on input I at time t .

B.2 Definitions

Definition B.2.1. *An automata with final states or terminating words F is known as an accepting automata if there is a set $A \subseteq F$ to be the accepting subset, thus making F/A a rejecting subset. An automata will either:*

- *Accepts if the automata enters $q \in A$*
- *Rejects if the automata enters $q \in F/A$*
- *Never halt if the automata never enters $q \in F$*

From formal language theory, the language of a machine is the set of inputs that the machine accepts and we talk about two machines accepting the same language. An input is accepted if it terminates by an accepting state or word.

With the automata we are concerned with, there are 3 potential outcomes: accepting, rejecting or the automata does not terminate (looping or perpetually running). We will introduce a stronger notion than accepting the same language. We call an automata able to represent another automata if the representing automata also rejects the same inputs.

Definition B.2.2. *Let $\text{accept}(M)$ and $\text{reject}(M)$ be the set of accepting and rejecting inputs to a automata M . An automata M_1 can be **represented** as a automata M_2 if both $\text{accept}(M_1) = \text{accept}(M_2)$ and $\text{reject}(M_1) = \text{reject}(M_2)$. That is, if we have an accepting input I_a to M_1 should also be an accepting input to machine M_2 . For a rejecting input I_r of M_1 , it should be that I_r be rejected by M_2 .*

Definition B.2.3. *Two types of automata are considered equivalent if every automaton of the first type has a corresponding representation in the second type, and vice versa.*

B.3 Proof that every MA can be represented by a TM

Lemma B.3.1. *For every MA there is a TM that represents it.*

For intuition for the TM that represents an arbitrary MA , we plan to use the states of the TM to store and read words. It will scan from left to right and append each read symbol to a word stored within the state. Once we have read x , it is stored with state q_x^{read} . This state transitions us to q_y^{write} such that $\delta(x) = (y, d)$. From this state, we write y by popping the final symbol until we have written the output word from right to left. As the last symbol of y is written, putting us at the first symbol which x would have been read, we use the output direction d to move left or write respectively. This puts the TM in an appropriate place to read the next input word.

Proof. Suppose the Mobile Automata is of the form: $MA = (n, \Sigma, \Gamma, \delta, b, F)$ and is accepting, so $\exists A \subseteq F$. We claim that an explicitly constructed Turing machine TM with a state q_ϵ^{read} exists such that for every t and input I either:

- If the MA has not halted by time t , $\text{tape}(MA, I, t) = \text{tape}(TM, I, (2n + 1)t)$ with the TM in a state q_ϵ^{read} and the head of the TM is in the same position as the head of the MA .

- If the MA has halted at time t to word $x \in F$, then this TM halt at time $(2n + 1)t + n$ in state q_x^{read} .

This accepting TM is constructed by $TM = (Q, \Sigma, \Gamma, \delta', q_0, b, F')$ such that

1. $Q = \{q_x^{\text{read}} \mid x \in \Gamma^i \text{ for } i \in \{1..n\}\} \cup \{q_{y_0\dots i}^{\text{write}} \mid \delta(x) = (y, d) \text{ for } i \in \{0\dots n-1\}\} \cup \{q_\epsilon^{\text{read}}\}$
2. $q_0 = q_\epsilon^{\text{read}}$
3. $F' = \{q_x^{\text{read}} \mid x \in F\}$, $A' = \{q_x^{\text{read}} \mid x \in A\}$
4. δ' such that

$$\begin{aligned} \delta' : \Gamma' \times Q &\rightarrow \Gamma' \times Q \times \{L, R\} \\ \delta'(\alpha, q_x^{\text{read}}) &= (\alpha, q_{x||\alpha}^{\text{read}}, R) \quad \text{if the length of } x \text{ is less than } n \\ \delta'(\alpha, q_x^{\text{read}}) &= (\alpha, q_{y_{1\dots n}}^{\text{write}}, L) \quad \text{if the length of } x \text{ is equal to } n \text{ and } \delta(x) = (y, d) \\ \delta'(\alpha, q_{y_{1\dots i}}^{\text{write}}) &= (y_i, q_{y_{1\dots(i-1)}}^{\text{write}}, L) \quad \text{if } i \text{ is greater than } 1 \\ \delta'(\alpha, q_{y_{1\dots i}}^{\text{write}}) &= (y_i, q_\epsilon^{\text{read}}, d) \quad \text{if } i \text{ is equal to } 1 \end{aligned}$$

We prove the claim by induction on time t . At $t = 0$, by the definition of TM we have $\text{tape}(TM, I, 0) = \text{tape}(MA, I, 0)$ and $q_0 = q_\epsilon^{\text{read}}$ with the head of the TM in the same position as the MA .

Now suppose the induction hypothesis, that $\text{tape}(MA, I, t) = \text{tape}(TM, I, (2n + 1)t)$ with the TM in state q_ϵ^{read} and the head is in the correct position.

- From q_ϵ^{read} the TM transitions right n times and reading a word x . The word is stored in the state as q_x^{read} .
- We transition from q_x^{read} to q_y^{write} such that $\delta(x) = (y, d)$ by moving 1 left without altering the symbol. This happens unless q_x^{read} is a halting state in which the TM has halted at time $(2n + 1)t + n$. By definition of F' , this would mean the MA must have halted at time t .
- q_y^{write} transitions left $n - 1$ times writing word y in reverse. Returning us to the initial position at q_ϵ^{read} .
- Depending on d , we move 1 left or right while writing y_0 .
- We return to q_ϵ^{read} , however offset left or right depending on d .

This then means that we have $\text{tape}(MA, I, t + 1) = \text{tape}(TM, I, (2n + 1)(t + 1))$ with the head of the TM in the same position as the MA , in q_ϵ^{read} . This concludes the inductive proof.

To finish we make sure if the MA accepts or rejects, so does the TM . This is proved as the TM accepts in state $q_x^{\text{read}} \in A'$ iff $x \in A$ by the definition of the TM .

Thus $\text{accept}(MA) = \text{accept}(TM)$ and $\text{reject}(MA) = \text{reject}(TM)$, meaning this TM represents this MA . $\square$

B.4 Proof that TM can be represented with an MA

Lemma B.4.1. *For every TM there is a MA that represents it.*

The intuition for the construction of the representing MA focuses on adding a component to the alphabet Γ , resulting in a new alphabet $\Gamma \times (Q \cup \{\kappa\})$. This new component is known as the 'state component' as it will hold information regarding the state. To add the component, we convert the input tape by replacing the input symbols with this new alphabet. The symbol κ signifies that the state component contains no information. Then, by construction, this MA is capable of reading $n = 2$ symbols at a time. We maintain that always one of these symbols will be κ , and the other will be a $q \in Q$ so we can always read a state on one of the two symbols read by the MA .

Proof. Suppose we have a Turing Machine: $TM = (Q, \Sigma, \Gamma, \delta, q_0, b, F)$.

We claim that an explicitly constructed MA exists such that for every t and input I either:

- If the TM has not halted by time t . There is a setup time s where the tape $\text{tape}(MA, I, t+s)$ is equal to $\text{tape}(TM, I, t)$ when we exclude the final component of each symbol on the tape, with the head in the same position. Also, the MA reads either $((\alpha, q), (\beta, \kappa))$ or $((\alpha, \kappa), (\beta, q))$.
- If the TM has halted at time t . There is a setup time s where $\text{tape}(TM, I, t)$ is the $\text{tape}(MA, I, t+s)$ with the state component omitted with the head of the MA in the same position of the TM reading either $((\alpha, q), (\beta, \kappa))$ or $((\alpha, \kappa), (\beta, q))$ and $q \in F$

We provide an explicit construction of a representing Mobile Automata: $MA = (n, \Sigma, \Gamma', \delta', b', F')$.

1. $n = 2$
2. $\Gamma' = (\Gamma \times (Q \cup \{\kappa\})) \cup \Sigma$
3. $b' = (b, \kappa)$
4. $F' = \{((x_1, q_1), (x_2, \kappa)) \in (\Gamma')^n \mid q_1 \in F\} \cup \{((x_1, \kappa), (x_2, q_2)) \in (\Gamma')^n \mid q_2 \in F\}$.
5. δ' such that

$$\delta' : (\Gamma')^n \rightarrow (\Gamma')^n \times \{L, R\}$$

Part 1 Input conversion

$$\delta'(x_1, x_2) = (((x_1, \kappa), (x_2, \kappa)), R) \text{ if } x_1, x_2 \in \Sigma$$

$$\delta'((x_1, \kappa), x_2) = (((x_1, \kappa), (x_2, \kappa)), R) \text{ where } x_2 \in \Sigma$$

$$\delta'((x_1, \kappa), b') = (((x_1, \kappa), b'), L)$$

$$\delta'((x_1, \kappa), (x_2, \kappa)) = (((x_1, \kappa), (x_2, \kappa)), L)$$

$$\delta'(b', (x_2, \kappa)) = ((b', (x_2, q_0)), R)$$

Part 2 TM transitions

$$\delta'((x_1, q_1), (x_2, \kappa)) = (((x_{out}, q_{out}), (x_2, \kappa)), L) \text{ if } \delta(x_1, q_1) = (x_{out}, q_{out}, L)$$

$$\delta'((x_1, \kappa), (x_2, q_2)) = (((x_{out}, q_{out}), (x_2, \kappa)), L) \text{ if } \delta(x_1, q_2) = (x_{out}, q_{out}, L)$$

$$\delta'((x_1, q_1), (x_2, \kappa)) = (((x_{out}, \kappa), (x_2, q_{out})), R) \text{ if } \delta(x_1, q_1) = (x_{out}, q_{out}, R)$$

$$\delta'((x_1, \kappa), (x_2, q_2)) = (((x_{out}, \kappa), (x_2, q_{out})), R) \text{ if } \delta(x_1, q_2) = (x_{out}, q_{out}, R)$$

First we find the starting time s will be given by the number of transitions in Part 1 of δ' , which converts the input Σ into Γ' . We assume the first symbol of the input is where the head of the TM will be placed, therefore having symbol b' to the left of it. After applying Part 1 of δ' , we will read the two symbols $((x_1, q_0), (x_2, \kappa))$ on the leftmost of the tape. Part 1 of δ' has no transitions which could halt.

We prove the claim by induction on time t . At $t = s$ MA has the head in the same position as the TM with symbol $((\alpha, q_0), (\beta, \kappa))$. $\mathbf{tape}(MA, I, s)$ is equal to $\mathbf{tape}(TM, I, t)$ when we exclude the state component of each symbol. This is as Part 1 of δ' appends the state component to each symbol, retaining the initial symbol and b' is b with the state component.

Suppose the induction hypothesis, that $\mathbf{tape}(MA, I, s + t)$ with the state component omitted is equal to $\mathbf{tape}(TM, I, t)$ and the head of the MA is in the same position as the TM . The MA also reads symbols $((\alpha, q), (\beta, \kappa))$ or $((\alpha, \kappa), (\beta, q))$.

As δ' conserves the amount of occurrences of κ across the whole tape. This means that there will always be one symbol that has state component $q \neq \kappa$. Furthermore, δ' ensures that this q will always be in the input at the next step of computation. For example writing $((x_{out}, q_{out}), (x_2, \kappa)), L$ leads to input $((y, \kappa), (x_{out}, q_{out}))$ when we move the head of the MA left. q_{out} is retained.

If $\mathbf{tape}(MA, I, s + t)$ reads $((\alpha, q), (\beta, \kappa))$ or $((\alpha, \kappa), (\beta, q))$ and $q \in F$, then by definition of F' we have halted.

Otherwise, consider the situation where the MA reads either the pair $((\alpha, q), (\beta, \kappa))$ or $((\alpha, \kappa), (\beta, q))$, with $q \notin F$. In this case, by the definition of the new set F' , we know the computation has not halted.

Have $\delta(\alpha, q) = (y, q_{out}, d)$. According to our construction of the new transition function δ' , the symbol α on the tape is replaced by the symbol y . Furthermore, depending on the direction of movement d , the state component is updated as follows:

- If the automaton moves right, then the state of the right-hand symbol in the pair (which could currently hold either q or κ) is updated to become q_{out} . - If the automaton moves left, then the state of the left-hand symbol in the pair is updated to become q_{out} .

In either case, the updated state q_{out} will remain stored for use in the subsequent step, ensuring the automaton's computation correctly continues according to the intended simulation.

Therefore, δ' moves us in the same direction as δ and replaces the non-state component with the relevant output symbol by δ . It must be that $\mathbf{tape}(MA, I, s + t + 1)$ removing the state component is equal to $\mathbf{tape}(TM, I, t + 1)$ with the head of the MA in the same position as the TM reading either $((\alpha, q), (\beta, \kappa))$ or $((\alpha, \kappa), (\beta, q))$ where q is the state output by δ . This is the end of the inductive proof.

To finish we make sure if the MA accepts or rejects, so does the TM . This is proved as the MA accepts if the symbol holding that state information: (α, q) has $q \in A$ iff $x \in A$ by the definition of the TM .

Thus, $\mathbf{accept}(MA) = \mathbf{accept}(TM)$ and $\mathbf{reject}(MA) = \mathbf{reject}(TM)$, meaning this TM represents this MA . This TM then represents the MA . $\square$